

HW2 protocol

Edward Student

Git: https://github.com/EdvardStudent23/apz_hw/tree/hazelcast

I have installed hazelcast and configured nodes like here:

Встановити і налаштувати Hazelcast <https://hazelcast.com/open-source-projects/downloads/>

Сконфігурувати і запустити 3 ноди (інстанси) об'єднані в кластер або як частину Java-застосування, або як окремі застосування <https://docs.hazelcast.com/hazelcast/5.3/getting-started/get-started-binary#step-6-scale-your-cluster>

Task 1-2:

I have started 3 hazelcast nodes using binary version which I installed:

3 nodes:

```
Member [127.0.0.1]:5701 - 30eee24b-3ce5-43ba-b037-ec08fb09735f this
]

cx# 06, 2026 12:22:49 - com.hazelcast.jet.impl.JobCoordinationService
INFO: [127.0.0.1]:5701 [dev] [5.6.0] Jet started scanning for jobs
cx# 06, 2026 12:22:49 - com.hazelcast.core.LifecycleService
INFO: [127.0.0.1]:5701 [dev] [5.6.0] [127.0.0.1]:5701 is STARTED
cx# 06, 2026 12:23:02 - com.hazelcast.internal.server.tcp.TcpServerConnection
INFO: [127.0.0.1]:5701 [dev] [5.6.0] Initialized new cluster connection between /127.0.0.1:5701 and /127.0.0.1:59030
cx# 06, 2026 12:23:02 - com.hazelcast.internal.cluster.ClusterService
INFO: [127.0.0.1]:5701 [dev] [5.6.0]

Members {size:2, ver:2} [
  Member [127.0.0.1]:5701 - 30eee24b-3ce5-43ba-b037-ec08fb09735f this
  Member [127.0.0.1]:5702 - fb98740a-8cbc-4d1a-b830-053367f911b8
]

cx# 06, 2026 12:23:07 - com.hazelcast.internal.server.tcp.TcpServerConnection
INFO: [127.0.0.1]:5701 [dev] [5.6.0] Initialized new cluster connection between /127.0.0.1:5701 and /127.0.0.1:59040
cx# 06, 2026 12:23:07 - com.hazelcast.internal.cluster.ClusterService
INFO: [127.0.0.1]:5701 [dev] [5.6.0]

Members {size:3, ver:3} [
  Member [127.0.0.1]:5701 - 30eee24b-3ce5-43ba-b037-ec08fb09735f this
  Member [127.0.0.1]:5702 - fb98740a-8cbc-4d1a-b830-053367f911b8
  Member [127.0.0.1]:5703 - 5fad8371-1462-4738-9cf8-33df62f3d38a
]
```

Hazelcast management system:

```
Windows PowerShell
at org.springframework.context.support.AbstractApplicationContext.refresh(AbstractApplicationContext.java:620) ~[spring-context-7.0.3.jar!/:7.0.3]
at org.springframework.boot.web.server.servlet.context.ServletWebServerApplicationContext.refresh(ServletWebServerApplicationContext.java:143) ~[spring-boot-web-server-4.0.2.jar!/:4.0.2]
at org.springframework.boot.SpringApplication.refresh(SpringApplication.java:756) ~[spring-boot-4.0.2.jar!/:4.0.2]
at org.springframework.boot.SpringApplication.refreshContext(SpringApplication.java:445) ~[spring-boot-4.0.2.jar!/:4.0.2]
at org.springframework.boot.SpringApplication.run(SpringApplication.java:321) ~[spring-boot-4.0.2.jar!/:4.0.2]
at org.springframework.boot.SpringApplication.run(SpringApplication.java:1365) ~[spring-boot-4.0.2.jar!/:4.0.2]
at org.springframework.boot.SpringApplication.run(SpringApplication.java:1354) ~[spring-boot-4.0.2.jar!/:4.0.2]
at com.hazelcast.webmonitor.MCApplication.main(MCApplication.java:63) ~[!/:?]
at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method) ~[?:?]
at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77) ~[?:?]
at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43) ~[?:?]
at java.base/java.lang.reflect.Method.invoke(Method.java:568) ~[?:?]
at org.springframework.boot.loader.launch.Launcher.launch(Launcher.java:106) [hazelcast-management-center-5.10.0.jar:?]
at org.springframework.boot.loader.launch.Launcher.launch(Launcher.java:64) [hazelcast-management-center-5.10.0.jar:?]
at org.springframework.boot.loader.launch.PropertiesLauncher.main(PropertiesLauncher.java:580) [hazelcast-management-center-5.10.0.jar:?]

2026-03-06 00:25:10,597 [ INFO] [main] [c.h.w.MCApplication]: Started MCApplication in 33.271 seconds (process running for 36.45)
2026-03-06 00:25:10,635 [ INFO] [main] [c.h.w.MCApplication]:
Hazelcast Management Center successfully started at http://localhost:8080/
```

Distribution of map after 1000 uploading:

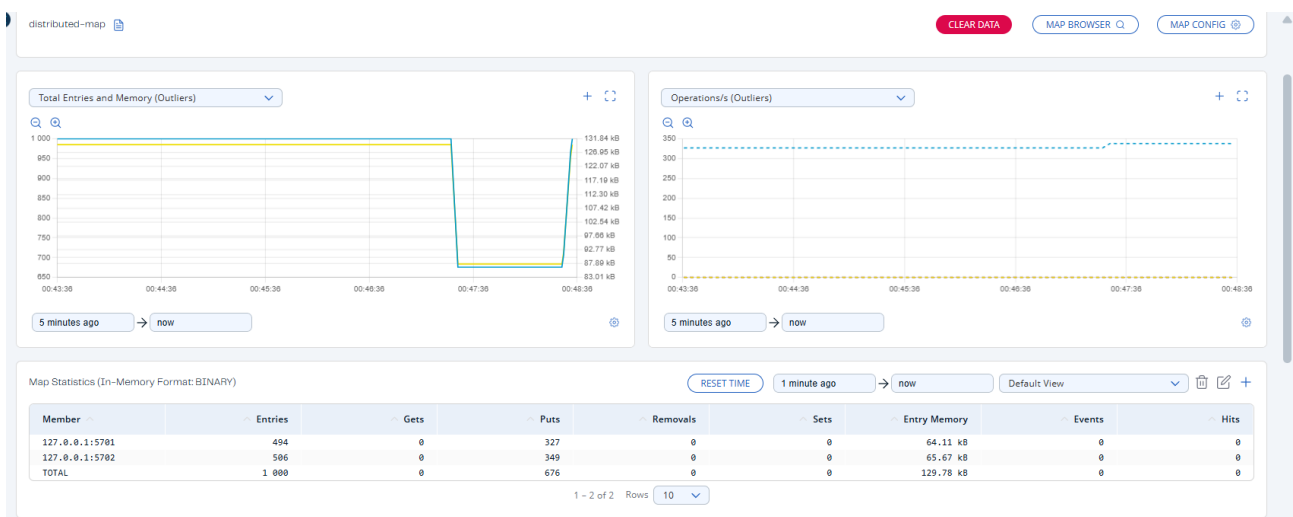
Map Statistics (In-Memory Format: BINARY)

RESET TIME 1 minute ago → now Default View

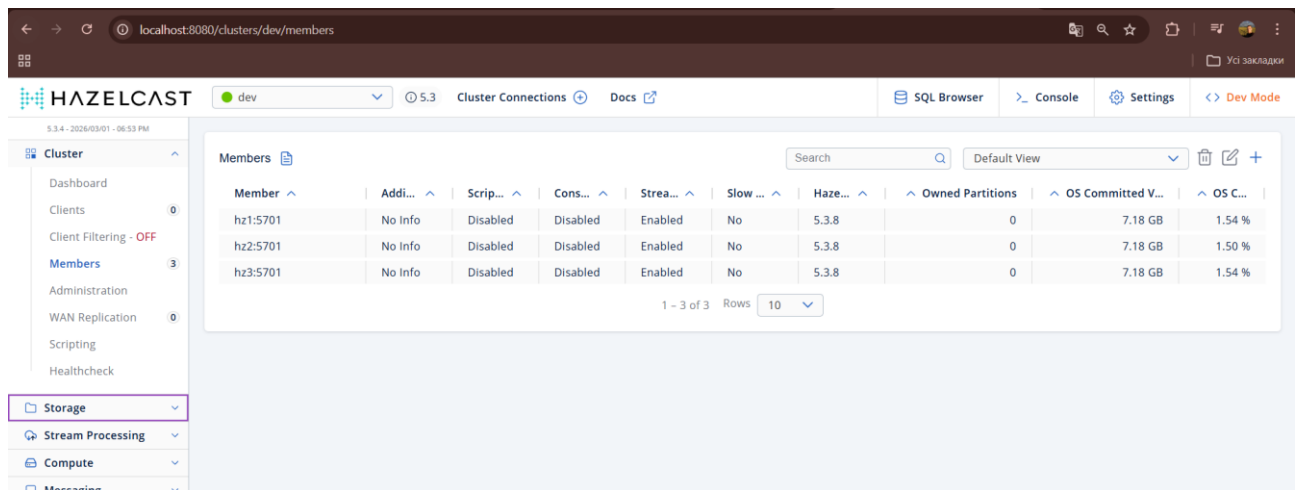
Member	Entries	Gets	Puts	Removals	Sets	Entry Memory	Events	Hits
127.0.0.1:5701	327	0	327	0	0	42.43 kB	0	0
127.0.0.1:5702	349	0	349	0	0	45.29 kB	0	0
127.0.0.1:5703	324	0	324	0	0	42.85 kB	0	0
TOTAL	1 000	0	1 000	0	0	129.78 kB	0	0

1 - 3 of 3 Rows 10

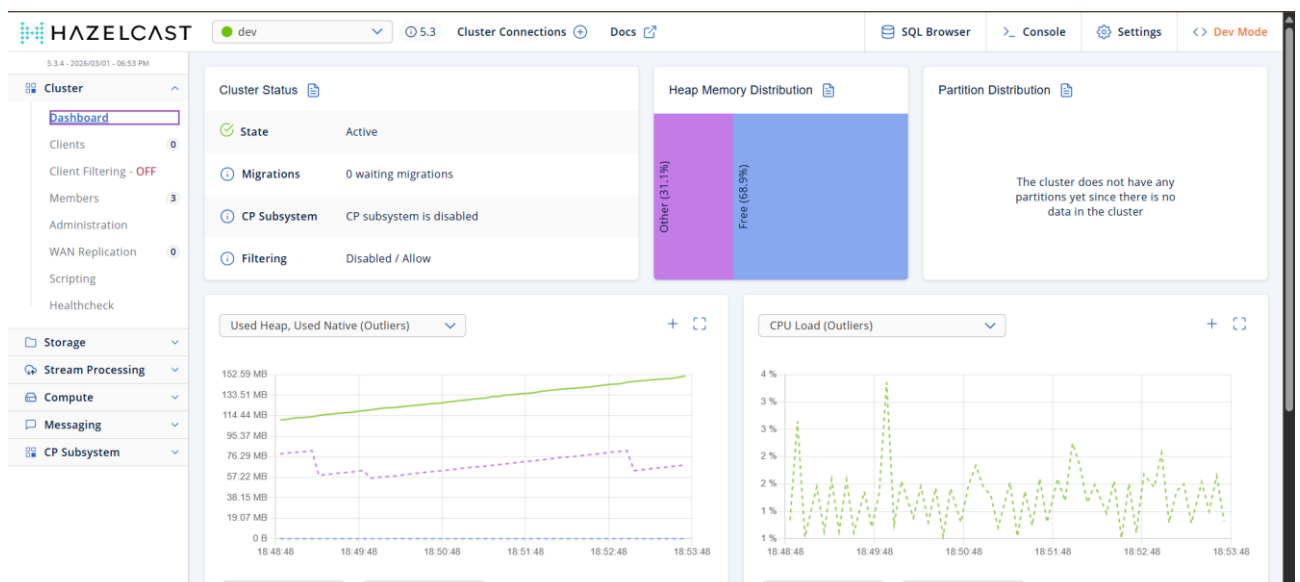
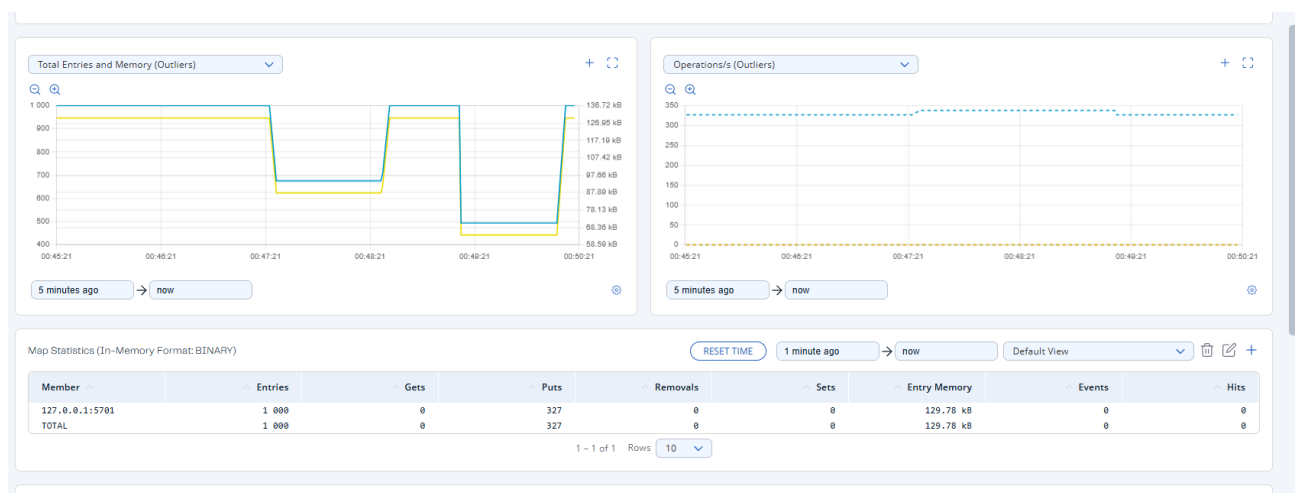
I have disabled 1 node:



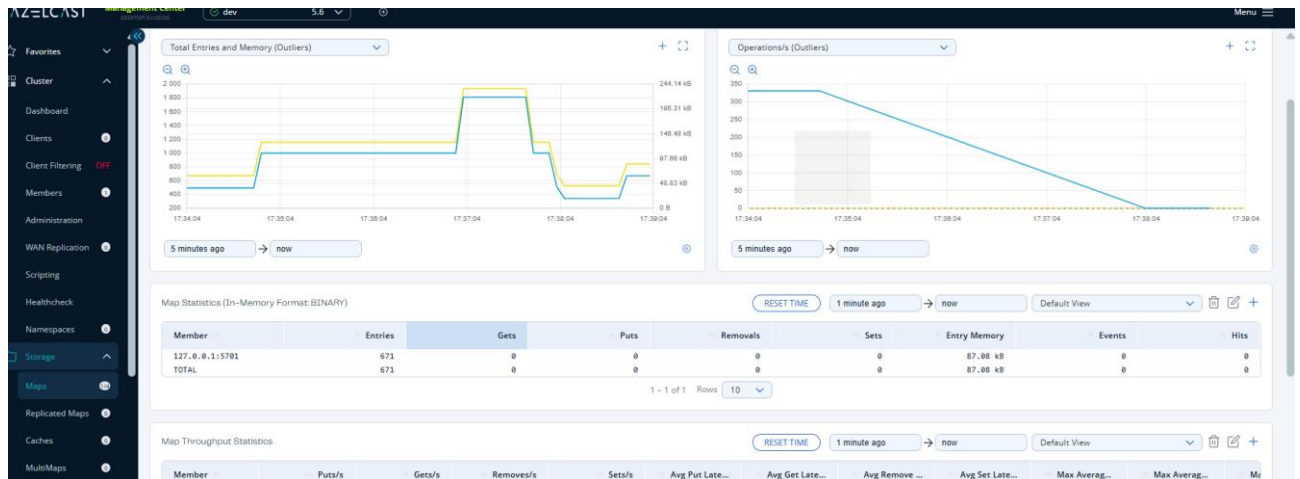
I have disabled the second node



3) I have disabled the 2nd node:



Now I have disabled 2 simultaneously. And I see the losing of data (671 < 1000 entries):



The task 4:

I have run the Task4.java

The experiment shows a final key value significantly lower than 30,000 because the code lacks locking mechanisms, leading to a classic race condition where concurrent updates are lost. Although the Management Center correctly records approximately 30,003 get and put operations—representing 10,000 iterations from each of the three clients plus initialization and final checks—many of these operations were redundant. Because multiple clients frequently read the same "old" value at the same time, they incremented it locally to the same "new" number and performed separate put requests that simply overwrote each other. So, finally I got the current value of 17401. This confirms that while the cluster faithfully processed every incoming network request, the lack of atomic synchronization caused the final data state to be inconsistent.

3 terminals:

```

PS C:\Users\user\Documents\3_course\sem2\apz\hw2> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users\user\AppData\Local\Temp\cp_3bbkeo4a8misw5mhun41wrpw.argfile' 'Task4' ...
e
INFO: Client statistics is enabled with period 5 seconds.
incrementation cycle 10 000
Finished in 5109 ms.
The current key value 12269
cxÉ. 07, 2026 5:59:05 11 com.hazelcast.core.LifecycleService
INFO: hz.client_1 [dev] [5.6.0] HazelcastC

PS C:\Users\user\Documents\3_course\sem2\apz\hw2> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users\user\AppData\Local\Temp\cp_3bbkeo4a8misw5mhun41wrpw.argfile' 'Task4' ...
ent.impl.statistics.ClientStatisticsService
e
INFO: Client statistics is enabled with period 5 seconds.
incrementation cycle 10 000
Finished in 4674 ms.
The current key value 16044
cxÉ. 07, 2026 5:59:06 11 com.hazelcast.core.LifecycleService

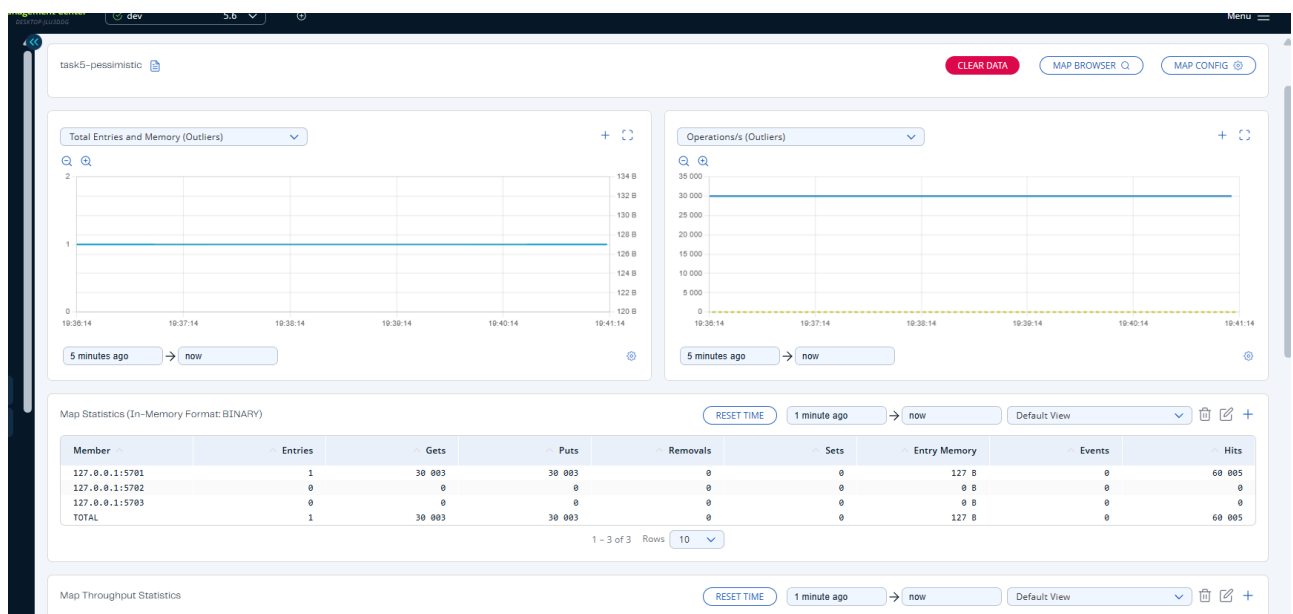
PS C:\Users\user\Documents\3_course\sem2\apz\hw2> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users\user\AppData\Local\Temp\cp_3bbkeo4a8misw5mhun41wrpw.argfile' 'Task4' ...
ent.impl.statistics.ClientStatisticsService
e
INFO: Client statistics is enabled with period 5 seconds.
incrementation cycle 10 000
Finished in 4040 ms.
The current key value 17401
cxÉ. 07, 2026 5:59:06 11 com.hazelcast.core.LifecycleService

```

Task 5:

3 terminals:

I ran the pessimistic locking experiment across three terminals and successfully reached the target value of 30,000, confirming that the race condition from the previous task was fully resolved. By following the documentation to implement `map.lock(key)` and `map.unlock(key)`, I forced the clients to wait for one another, ensuring every single increment was recorded correctly without any updates being lost. While my terminals finished with different intermediate values like 29,799 and 29,114, these simply represented the state of the counter at the exact moment those specific clients finished their loops while the others were still committing their final updates. I noticed that the execution time increased significantly to approximately 22,000 ms, which highlights the performance trade-off required to achieve absolute consistency in a distributed environment.



Task 6:

I completed the Optimistic Locking task using three concurrent terminals, achieving the target value of 30,000 with a significant performance improvement.

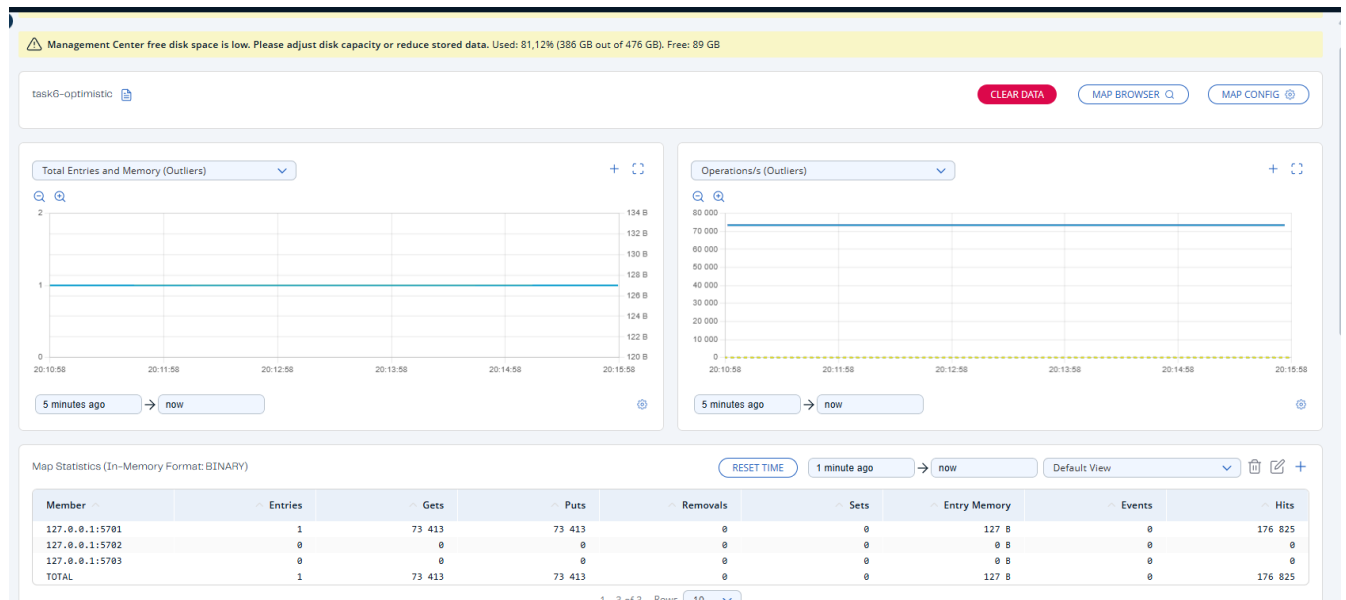
By using the `map.replace(key, oldValue, newValue)` method, the clients finished their loops in approximately 7,300–7,900 ms. This is nearly three times faster than the pessimistic approach, fulfilling the "performance boost" promised in the documentation for low-lock environments. While each terminal finished at different times, the final counter correctly reached the cumulative total of 30,000.

The Management Center statistics reveal that this speed comes from high retry traffic. Because the clients were competing for the same key, many replace calls failed when the oldValue had already been changed, forcing a retry. This resulted in 73,413 Gets and Puts—more than double the original operation count

```
ent.impl.statistics.ClientStatisticsService
INFO: Client statistics is enabled with period 5 seconds.
Starting optimistic lock increment...
Finished in 7308 ms.
Final value: 22265
cxÉ. 07, 2026 7:51:17 11 com.hazelcast.cor

INFO: Client statistics is enabled with period 5 seconds.
Starting optimistic lock increment...
Finished in 7936 ms.
Final value: 28123
cxÉ. 07, 2026 7:51:18 11 com.hazelcast.cor
e.LifecycleService
INFO: hz.client_1 [dev] [5.6.0] HazelcastC

INFO: Client statistics is enabled with period 5 seconds.
Starting optimistic lock increment...
Finished in 7674 ms.
Final value: 30000
cxÉ. 07, 2026 7:51:19 11 com.hazelcast.cor
e.LifecycleService
```



Task 7:

Data Integrity Comparison

- No Locking (Task 4): This implementation resulted in significant data loss, yielding a final value of only 17,401 instead of the expected 30,000. This confirms that without atomic synchronization, concurrent updates overwrite each other in a classic race condition.
- Pessimistic and Optimistic Locking (Tasks 5, 6): Both implementations successfully reached the target value of 30,000. These methods ensured that every single increment was recorded correctly, resolving the consistency issues found in the non-locked version.

Performance Comparison: Pessimistic vs. Optimistic

- Pessimistic Locking: This was the slowest approach, taking approximately 22,000 ms to complete. The high execution time is due to the performance tradeoff of forcing clients to wait in a queue for a network-based lock.
- Optimistic Locking: This approach was significantly faster, finishing in approximately 7,300–7,900 ms. This is nearly three times faster than the pessimistic approach.

Task 8:

1. I have created a new config file for the Queue.

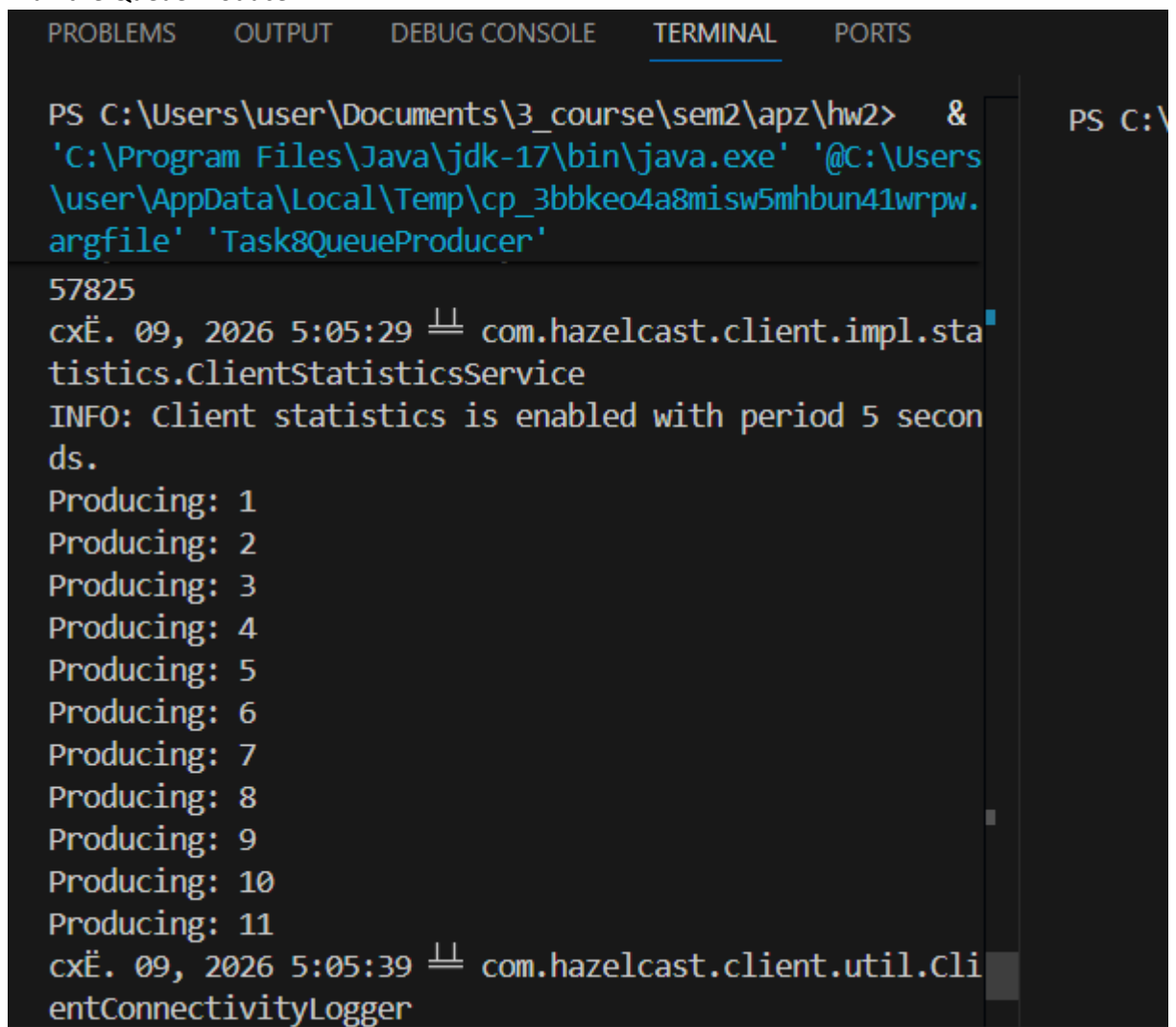
2. I changed the original config file to my custom file (your path can be different from my ones):
Copy-Item -Path "C:\Users\user\Documents\3_course\sem2\apz\hw2\hazelcast.xml" -Destination "C:\Users\user\Documents\3_course\sem2\apz\hw2_hazelcast_sets\hazelcast-5.6.0\config\hazelcast.xml" -Force

3. I cleaned environmental variables:
\$env:HAZELCAST_CONFIG=""
\$env:JAVA_OPTS=""

4. Restarted all the nodes:

```
cd "C:\Users\user\Documents\3_course\sem2\apz\hw2_hazelcast_sets\hazelcast-5.6.0\bin\"  
./hz-start.bat
```

5. I run the Queue Producer:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\user\Documents\3_course\sem2\apz\hw2> &  
'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users  
\user\AppData\Local\Temp\cp_3bbkeo4a8misw5mhbun41wrpw.  
argfile' 'Task8QueueProducer'  
57825  
cxĚ. 09, 2026 5:05:29 11 com.hazelcast.client.impl.sta  
tistics.ClientStatisticsService  
INFO: Client statistics is enabled with period 5 secon  
ds.  
Producing: 1  
Producing: 2  
Producing: 3  
Producing: 4  
Producing: 5  
Producing: 6  
Producing: 7  
Producing: 8  
Producing: 9  
Producing: 10  
Producing: 11  
cxĚ. 09, 2026 5:05:39 11 com.hazelcast.client.util.Cli  
entConnectivityLogger
```

6. Next I run 2 Queue Consumers. When consumers started to consume:

```

ask4.java
ask5.java
ask6.java
ask8QueueConsumer...
ask8QueueProducerj...

PS C:\Users\user\Documents\3_course\sem2\apz\hw2> &
'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users\
\user\AppData\Local\Temp\cp_3bbke04a8misw5mhbn41wrpw.
argfile' 'Task8QueueProducer'

Producing: 12
Producing: 13
Producing: 14
Producing: 15
Producing: 16
Producing: 17
Producing: 18
Producing: 19
Producing: 20
Producing: 21
Producing: 22
Producing: 23
Producing: 24
Producing: 25
Producing: 26
Producing: 27
Producing: 28
Producing: 29
Producing: 30
Producing: 31
Producing: 32
Producing: 33
Producing: 34
Producing: 35

INFO: hz.client_1 [dev] [5.6.0] Authenticated with ser
ver [127.0.0.1]:5703:c5e7ed82-b5c7-4136-8450-9fb31497e
9e6, server version: 5.6.0, local address: /127.0.0.1:
50795
cxÉ. 09, 2026 5:07:00 11 com.hazelcast.client.impl.sta
tistics.ClientStatisticsService
INFO: Client statistics is enabled with period 5 secon
ds.
Consumed: 13
Consumed: 14
Consumed: 17
Consumed: 18
Consumed: 21
Consumed: 23
Consumed: 24
Consumed: 26
Consumed: 28
Consumed: 30
Consumed: 33
Consumed: 34
Consumed: 36
Consumed: 38
Consumed: 40
Consumed: 43
Consumed: 45
Consumed: 47
Consumed: 48
Consumed: 50
Consumed: 53

9e6, server version: 5.6.0, local address: /127.0.0.1:
50792
cxÉ. 09, 2026 5:06:59 11 com.hazelcast.client.impl.sta
tistics.ClientStatisticsService
INFO: Client statistics is enabled with period 5 secon
ds.
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Consumed: 11
Consumed: 12
Consumed: 15
Consumed: 16
Consumed: 19
Consumed: 20
Consumed: 22
Consumed: 25
Consumed: 27
Consumed: 29
Consumed: 31
Consumed: 32
Consumed: 35

PS C:\Users\user\Documents\3_course\sem2\apz\hw2> &
'C:\Program Files\Java\jdk-17\bin\java.exe' '@C:\Users\
\user\AppData\Local\Temp\cp_3bbke04a8misw5mhbn41wrpw.
argfile' 'Task8QueueProducer'

Producing: 79
Producing: 80
Producing: 81
Producing: 82
Producing: 83
Producing: 84
Producing: 85
Producing: 86
Producing: 87
Producing: 88
Producing: 89
Producing: 90
Producing: 91
Producing: 92
Producing: 93
Producing: 94
Producing: 95
Producing: 96
Producing: 97
Producing: 98
Producing: 99
Producing: 100
Producer Finished!
cxÉ. 09, 2026 5:07:05 11 com.hazelcast.core.LifecycleS
ervice

Consumed: 50
Consumed: 53
Consumed: 54
Consumed: 57
Consumed: 58
Consumed: 61
Consumed: 63
Consumed: 64
Consumed: 66
Consumed: 68
Consumed: 70
Consumed: 73
Consumed: 74
Consumed: 76
Consumed: 79
Consumed: 81
Consumed: 82
Consumed: 85
Consumed: 87
Consumed: 89
Consumed: 91
Consumed: 92
Consumed: 95
Consumed: 96
Consumed: 99
cxÉ. 09, 2026 5:07:10 11 com.hazelcast.client.util.Cli
entConnectivityLogger
INFO: hz.client_1 [dev] [5.6.0]

Consumed: 52
Consumed: 55
Consumed: 56
Consumed: 59
Consumed: 60
Consumed: 62
Consumed: 65
Consumed: 67
Consumed: 69
Consumed: 71
Consumed: 72
Consumed: 75
Consumed: 77
Consumed: 78
Consumed: 80
Consumed: 83
Consumed: 84
Consumed: 86
Consumed: 88
Consumed: 90
Consumed: 93
Consumed: 94
Consumed: 97
Consumed: 98
Consumed: 100
cxÉ. 09, 2026 5:07:09 11 com.hazelcast.client.util.Cli
entConnectivityLogger
INFO: hz.client_1 [dev] [5.6.0]

```

2. Producer Behavior (Queue Full)

I ran a single Producer to write values 1 through 100.

- 1) The Producer printed Producing: 1 through Producing: 11 and then blocked (stalled).
- 2) Analysis: Because the queue limit was 10, the 11th put() operation was forced to wait. This confirms that when the queue is full and no consumers are present, the producer thread is suspended until space becomes available.
- 3) I started two concurrent Consumer clients to drain the queue.
- 4) As soon as the consumers started, the Producer resumed. The two consumers split the workload (e.g., Consumer 2 took items 1-12, while Consumer 1 took 13-14).

Reading Pattern: The values were read using the Competing Consumers pattern. Each message was processed immediately and only by one of the two clients. No data was duplicated or lost.

There is zero overlap between the two terminals. For example, while Consumer 1 (left) processed items 50, 53, and 54, Consumer 2 (right) was simultaneously processing items 52, 55, and 56. Once an item is taken by one client, it is instantly removed from the distributed queue, making it unavailable to the other.

How are values read by two clients?

They are distributed on a firstcome, firstserved basis. Hazelcast ensures that while multiple clients can listen to the same queue, each individual item is delivered to only one consumer, effectively balancing the processing load.

Write behavior when full?

The `IQueue.put()` method is a blocking operation. It prevents the producer from overfilling the queue, providing natural backpressure in the system.